



RangeGuard: Efficient, Bounded Approximate Error Correction for Reliable DNNs

Hanum Ko*, Sangheum Yeon*, Jong Hwan Ko†, and Junrae Kim†

Sungkyunkwan University, Suwon, Republic of Korea

*Dept. of Semiconductor Convergence Engineering †Dept. of Electrical and Computer Engineering
{rhgksdma, shhj9787, jhko, dale40}@skku.edu

Abstract—As DRAM scales in density and adopts 3D integration, raw fault rates increase and multi-bit errors are no longer rare. Such errors can severely impact *Deep Neural Networks (DNNs)*: although DNNs tolerate small numerical perturbations, random bit flips can create extreme outliers that propagate and sharply degrade accuracy. *Large Language Models (LLMs)* are particularly vulnerable because attention, residual, and normalization layers can amplify and preserve a single corrupted activation across many layers, destabilizing inference.

This paper introduces *RangeGuard*, a metadata-centric error-correcting framework that provides strong reliability and high efficiency based on *bounded approximate correction*. Instead of protecting raw bits, RangeGuard encodes compact *Range Identifiers (RIDs)* that capture the numerical range of each value. These compact metadata enable efficient use of limited redundancy and concentrate protection on range changes—which indicate harmful semantic deviations—while ignoring benign intra-range variations. Upon detecting a range change, RangeGuard restores the correct range and substitutes a representative value, ensuring that error magnitudes are bounded within the range. Based on RIDs, RangeGuard can tolerate 64+ bits of error using only 16 bits of parity available in GPU memories without a noticeable accuracy loss. By introducing semantic range protection, RangeGuard enables reliable DNN execution even under frequent memory errors and tight redundancy budgets.

Index Terms—DNN reliability, memory errors, error correction code, HBM, approximate error correction

I. INTRODUCTION

DRAM reliability is degrading with density/voltage scaling and 3D integration. Conventional 2D devices (e.g., DDR4/5) [1], [2] face shrinking noise margins and rising marginal cell defects. High Bandwidth Memory (HBM) further introduces complex processing (e.g., wafer thinning, Through Silicon Via (TSV) formation, thermal cycling) and harsher thermal/mechanical stress, which elevate fault susceptibility. Fleet-scale reports find HBM2 raw error rates substantially higher than DDR4 ($\sim 500\times$), and production GPU clusters observe uncorrectable multi-bit HBM3 errors on the order of hours [3], [4]. As systems provision more memories for bandwidth and capacity, dense multi-bit faults are no longer rare-event tails.

These faults can be devastating for *Deep Neural Networks (DNNs)*. While DNNs tolerate small numerical perturbations—such as quantization noise or rounding error that remains a tiny fraction of the original value—random *bit flips* can cause catastrophic deviations. For example, flipping the most significant exponent bit of a *bfloat16 (BF16)* value turns

0.5 into $2^{127} \approx 1.7 \times 10^{38}$, injecting an astronomically large outlier that can propagate through the network and sharply degrade accuracy.

Large Language Models (LLMs) are particularly susceptible. Unlike *Convolutional Neural Networks (CNNs)*, where spatial filters may attenuate local anomalies, a transformer in an LLM can *amplify* a single corrupted token feature through multi-head attention, *preserve* it across layers via residual connections, and *stabilize* its presence through normalization. This combination lets one faulty activation persist, interact with many tokens, and cascade through subsequent blocks, magnifying its impact on logits and sequence generation, and ultimately destabilizing the final output (Section III-C).

However, providing robust protection under realistic system constraints is challenging. DNN accelerators (e.g., GPUs) operate on small memory-access granularities (e.g., 32 bytes) to avoid overfetch, leaving little opportunity to amortize redundancy. Contemporary GPU memories provision only 2 bytes of parity per 32-byte block (6.25%), yet practical failure modes can corrupt up to 4 bytes within the same block [5]. Under this tight budget, conventional *bit-centric* protection schemes struggle to handle dense multi-bit corruptions without incurring unacceptable storage or bandwidth overheads. These constraints motivate rethinking how to spend scarce parity so that it protects what actually matters for model correctness.

This paper introduces *RangeGuard*, a metadata-centric error-correcting framework for DNNs and approximate computing. Instead of protecting raw bits, RangeGuard protects per-value *metadata*: a compact *Range Identifier (RID)* that encodes the value’s numeric interval. This design enables far more efficient use of limited redundancy: for example, restoring a corrupted 32-bit value from a dense 32-bit fault can require at least 64 bits of redundancy, whereas restoring a 4-bit RID of that value needs only 8 bits. Moreover, RangeGuard allocates its correction capability to errors that *change ranges*. If bit flips perturb only lower bits and keep the value within the same range, the RID is unchanged and consumes no correction budget. Only severe errors that flip critical bits and cause an *inter-range* deviation trigger correction, ensuring that scarce redundancy is spent on semantically harmful faults rather than benign intra-range perturbations.

Figure 1 presents an overview of RangeGuard. On a write, each value is mapped to an RID via a *RangeMap*, and an *Error Correcting Code (ECC)* encodes the RID symbols into a short

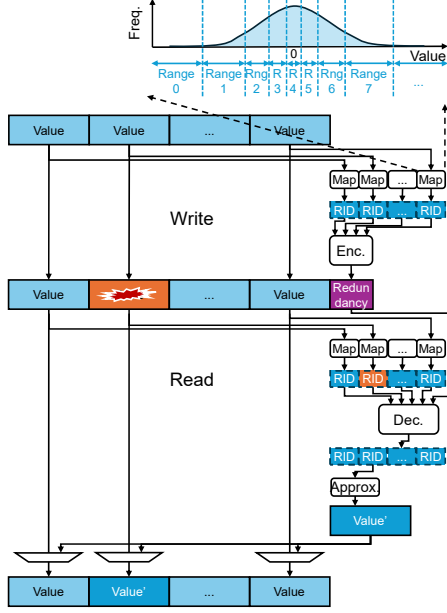


Figure 1. An overview of RangeGuard. ECC operates on RIDs, rather than raw data bits, and repairs corrupted data by restoring it to a representative value of the range.

redundancy field. The explicit RIDs are then discarded; only the original data and the encoded RID redundancy are stored in memory. On a read, RangeGuard regenerates candidate RIDs from the fetched data and applies ECC decoding over these regenerated RIDs and the stored redundancy. Inter-range errors are detected and corrected by the decoder, whereas intra-range errors remain undetected and do not consume correction capacity. After restoring the RID, RangeGuard replaces the corrupted value with a representative value for that range, so the resulting error magnitude is bounded by the range width and has limited impact on DNN accuracy.

Because each RangeMap is tailored to a specific data format and value distribution, a single map offers limited flexibility in modern DNN systems with multiple formats and diverse value distributions. To address this issue, RangeGuard utilizes multiple maps and address-based ECC selection [6]. The memory controller maintains multiple maps for different numeric formats or value distributions while also preserving a conventional exact-correction scheme. The operating system supports the appropriate protection mode for each data region by encoding ECC-selection metadata (*Map Tag*) in otherwise unused upper physical-address bits (e.g., bits [57:54]). Since these bits lie beyond the implemented physical-memory capacity, they do not affect the actual memory location and instead serve only as control metadata. This mechanism enables flexible per-region protection without disrupting normal address translation or existing ECC operation.

Our evaluation shows that RangeGuard delivers strong robustness to dense, multi-bit corruptions within the standard redundancy budget. Using the 2-byte redundancy per 32-byte block available in contemporary GPU memories, one RangeGuard instantiation tolerates two inter-range errors affecting up

to 64 data bits, while bypassing an unbounded number of intra-range errors. This strong protection preserves LLM accuracy even at extremely high *Bit Error Rates (BERs)*: for example, an unprotected Llama-3.2-1B model suffers significant accuracy loss from just a single bit flip at critical positions, whereas RangeGuard maintains robust execution up to $BER = 10^{-6}$ and incurs only small degradation at $BER = 10^{-5}$. At the same time, RangeGuard operates within existing GPU memory interfaces and incurs negligible performance overhead and no additional storage overhead.

The major contributions of this paper are as follows:

- We propose RangeGuard, a metadata-centric protection framework that uses per-value Range Identifiers (RIDs) to provide strong, efficient reliability under tight redundancy budgets.
- We analyze how DRAM bit flips map to value errors and quantify how these errors degrade end-to-end accuracy for both CNNs and LLMs, revealing LLMs are highly vulnerable to errors possibly due to their transformer layers.
- We develop an optimization framework for constructing the value-to-RID range mapping and present practical heuristics that minimize its overhead while preserving protection quality.
- We design a memory system that supports diverse data formats and value distributions with RangeGuard.

The rest of this paper is organized as follows. Section II provides background on numeric formats and ECC. Section III characterizes how bit flips translate into value errors and end-to-end accuracy loss. Section IV discusses prior work on DNN fault tolerance and ECC. Section V presents the design of RangeGuard, including RangeMap construction and architectural support. Section VI evaluates RangeGuard’s reliability and overheads. Section VII concludes the paper.

II. BACKGROUND

This section lays the groundwork for RangeGuard by reviewing DNN data types and ECC fundamentals.

A. DNN Data Types

DNNs trade accuracy, throughput, memory footprint, and energy through their numeric formats. Two families dominate: *floating point* (for wide dynamic range and optimization stability) and *integer* (for efficient deployment via quantization).

1) *Floating-point (FP)*: Floating-point formats capture fine-grained value changes while offering large dynamic range. Although FP32 (32-bit) was the historical default, modern training is largely mixed precision: FP16 (16-bit) or BF16 (*bfloat16*, 16-bit) for activations/gradients, with FP32 accumulators or “master” weights. FP16 typically employs dynamic loss scaling to avoid underflow, whereas BF16 retains FP32’s exponent range to reduce the risk of underflow and improve training stability. Emerging 8-bit floating-point formats (FP8) typically use 4-bit or 5-bit exponents (E4M3 and E5M2, respectively), enabling lower storage, memory traffic, and computation cost.

2) *Integers (INT)*: After training, many systems *quantize* tensors to low-bit *integers* for efficient inference. Quantization maps real values to integers via a scale (and optionally a zero point). A widely used baseline employs INT8 weights and activations with INT32 accumulation; pushing below 8 bits (e.g., INT4/INT3) generally requires quantization-aware training and related techniques to preserve accuracy.

Quantization leverages DNNs’ tolerance to small perturbations: under uniform quantization, the absolute error is bounded by the quantization step size and is typically only a small fraction of the original value, yielding negligible accuracy loss. In contrast, a *bit flip* can cause far larger deviation than the original value. For floating point, flipping exponent bit p scales the value by $2^{\pm 2^p}$, exhibiting an effectively *double-exponential* sensitivity to bit position (Section III-B). Such deviations beyond the original value can be intolerable for DNNs (Section III-C), motivating strong protection against memory errors.

B. Error Correcting Codes

Error Correcting Codes (ECC) can detect and correct errors by adding redundancy to data, forming a codeword [6]–[10]. A common baseline is Single-Error Correction, Double-Error Detection (SEC–DED), which corrects any single-bit error and detects (but cannot correct) any double-bit error in a codeword.

DRAM errors often appear as *bursts* aligned with the device’s internal organization (e.g., I/O pins, sub-wordline). In such settings, non-binary, *symbol*-based ECC can be more efficient than bit-oriented codes: by grouping a spatially clustered error into one symbol, the decoder repairs the entire cluster as a single unit, reducing the redundancy needed for a given level of protection.

Reed–Solomon (RS) [11] codes are a canonical family of symbol-based codes. An $RS(n, k)$ code protects k data symbols with $(n - k)$ parity symbols to form an n -symbol codeword. The decoder can correct up to $\lfloor (n - k)/2 \rfloor$ arbitrary symbol errors, provided the code length satisfies $n \leq 2^m - 1$, where m is the symbol size. In practice, memory word sizes rarely match the full code length, so systems use *shortened* RS codes: some leading symbol positions are conceptually fixed (not stored/transmitted), and encoding/decoding proceed as usual over the remaining positions. Shortening preserves the same error-correction capability for the active symbols; corrections in the omitted positions are reported as error detections instead of silent miscorrections.

RangeGuard uses two RS configurations: (1) a 4-bit $RS(12, 8)$, providing *Double-Symbol Correction (DSC)* for 4-bit RID symbols, and (2) an 8-bit $RS(10, 8)$, providing *Single-Symbol Correction (SSC)* for 8-bit RID symbols. Both consume exactly 16 parity bits per 256-bit block, matching the HBM budget.

III. MOTIVATION

This section explains how low-level bit flips can propagate to large value deviations in DNN tensors and ultimately degrade model accuracy.

A. HBM Errors

HBM underpins modern AI infrastructure by delivering extreme bandwidth and high energy efficiency via TSVs and silicon interposers. The same 2.5D integration, however, introduces—and amplifies—reliability hazards. Dense stacking, TSV/microbump interconnects, and elevated operating temperatures increase the likelihood of manufacturing defects, thermo-mechanical stress, and heat-induced marginalities compared to planar DDR devices [12]–[20].

Fleet-scale measurements demonstrate a pronounced reliability gap between stacked HBMs and traditional DRAMs [3], [4], [21]–[23]. A useful comparison is HBM2 versus DDR4, because both expose *raw* device errors (i.e., no on-die ECC). In ByteDance’s server fleet (100,000 servers with DDR4 memories), the average incidence over eight months is *0.07 errors per device per month* [3]. In contrast, Huawei’s study of 15,000 domain-specific accelerators over two years reports a mean of *35 errors per device per month* for HBM2 core dies [4]. While platform and workload differences preclude a perfectly controlled comparison (HBM with higher bandwidth can exhibit more frequent errors from a permanent fault), the resulting $\sim 500\times$ gap highlights the severity of HBM reliability challenges.

Later generations (e.g., HBM3 and DDR5) integrate *on-die ECC (O-ECC)* that corrects many errors before they exit the bank group, reducing externally visible errors. Public descriptions indicate that HBM3 employs stronger device-internal coding than DDR5 (e.g., 16-bit symbol correction with 12.5% internal redundancy versus single-bit correction with 6.25% in DDR5), targeting multi-bit patterns that arise from peripheral faults [5], [24], [25].

However, the scope of O-ECC is inherently limited: faults originating *outside* the bank group—global data/command lines, I/O circuitry, and TSVs/microbumps—can still surface at the system boundary. Moreover, certain *intra*-bank peripheral failures can exceed the 16-bit correction capability. For example, a HBM3 sub-wordline (SWL) driver typically fans out to 32 data bits per access and operates at elevated voltages to suppress leakage [5]; this stress raises wear-out risk, and a single defective driver can corrupt all serviced bits in a burst. Such 32-bit errors overwhelm O-ECC’s correction capability and propagate to the host.

As a result, data centers continue to observe non-trivial *exposed* HBM errors. During a 54-day Llama-3 (405B) pre-training run on 16,384 H100 GPUs, Meta reported 72 unexpected job interruptions attributed to uncorrectable HBM3 errors [23]. The GPUs utilize processor-side ECC (*System ECC, or S-ECC*) with Single Error Correction (SEC) capability, and this number indicates that the cluster experienced exposed multi-bit errors every 18 hours.

Although differences in platforms and measured metrics (e.g., job interruptions versus error counts) prevent a perfectly controlled comparison, this reported failure rate represents a substantial improvement over HBM2 (from 5,600 FIT/device to 90 FIT/device). However, the overall uncorrectable error

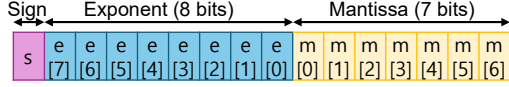


Figure 2. The bfloat16 (BF16) data format.

rate remains significantly higher than that of DDR4 (near zero FIT), underscoring the considerable reliability challenges in recent HBMs.

B. Memory Errors to Value Errors

Not all memory errors translate into the same numerical impact: the magnitude of a *value error* depends on (i) the data type, (ii) flipped bit location, and (iii) the original value. We illustrate this sensitivity with single-bit flips on BF16, while RangeGuard also considers multi-bit faults and other data types.

A BF16 number has one sign bit s , eight exponent bits e (bias = 127), and seven mantissa (fraction) bits m (Figure 2). For a normalized value,

$$x = (-1)^s \times 2^{e-\text{bias}} \times (1.m), \quad (1)$$

where $(1.m)$ denotes the implicit leading-1 mantissa.

1) *Sign-bit flip*: Toggling the sign ($s \rightarrow 1-s$) yields $x' = -x$ and an absolute error $|x' - x| = 2|x|$. This error can be large but is *bounded* and strictly proportional to the original magnitude.

2) *Mantissa-bit flip*: Flipping the k -th mantissa bit (weight $2^{-(k+1)}$, with $k=0$ the most significant) produces

$$x' = (-1)^s 2^{e-\text{bias}} ((1.m) \pm 2^{-(k+1)}) = x \left(1 \pm \frac{2^{-(k+1)}}{1.m} \right)$$

$$|x' - x| = \frac{2^{-(k+1)}}{1.m} \times |x| \leq \frac{1}{2} |x|. \quad (2)$$

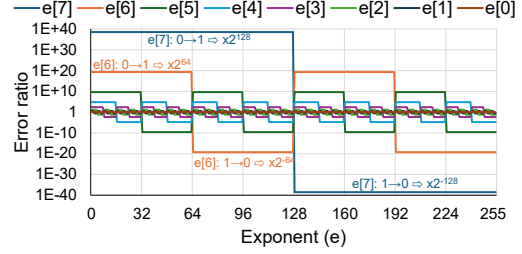
Thus a *single* mantissa-bit flip cannot exceed half the original magnitude (tight when $m=0$, $k=0$), and its effect decays exponentially with k . Because these contributions form a geometric series, the cumulative perturbation from mantissa errors remains bounded by the original value (i.e., $|x' - x| < |x|$) even if *all* mantissa bits flip.

3) *Exponent-bit flip*: Exponent flips can be catastrophic. Flipping the p -th exponent bit (weight 2^p ; with $p=0$ the least significant) changes the stored exponent by $\Delta e = \pm 2^p$, giving

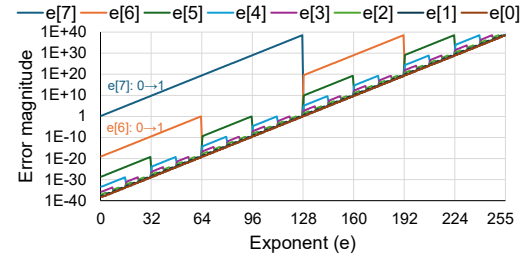
$$x' = (-1)^s \times 2^{e+\Delta e-\text{bias}} \times (1.m) = 2^{\Delta e} \times x. \quad (3)$$

This has three implications. First, the *Scale Factor* ($SF = 2^{\Delta e} = 2^{\pm 2^p}$) grows *double-exponentially* with bit index, so high-order flips cause orders-of-magnitude shifts. Second, error *polarity* matters: $0 \rightarrow 1$ amplifies ($SF = 2^{+2^p} \geq 2$) and can yield effectively *unbounded* error (e.g., $p=7$ gives $SF = 2^{128} \approx 3 \times 10^{38}$), whereas $1 \rightarrow 0$ attenuates ($SF = 2^{-2^p} \leq \frac{1}{2}$), pulling the value towards zero and producing a *bounded* error not exceeding $|x|$.

To visualize SFs, we inject single-bit flips across different bit positions of specific exponent values e and plot the error



(a) Error ratio ($=x'/x$) under a single exponent-bit flip.



(b) Error magnitude ($=|x' - x|$) for the same scenarios.

Figure 3. Impact of exponent-bit flips in BF16. We vary the original exponent value e and its flip bit position $e[i]$.

Table I
SUMMARY OF ERROR RATIOS ($=x'/x$) INDUCED BY BIT FLIPS.

Bit Flip	BF16 Bit Position						
	s	$e[7]$	$e[6]$	$e[5]$	$e[0]$	$m[0]$	$m[1]$
$0 \rightarrow 1$	$\times (-1)$	$\times 2^{2^7}$	$\times 2^{2^6}$	$\times 2^{2^5}$	$\times 2^{2^0}$	$\times 2^{-1}$	$\times 2^{-2}$
$1 \rightarrow 0$	$\times (-1)$	$\times 2^{-2^7}$	$\times 2^{-2^6}$	$\times 2^{-2^5}$	$\times 2^{-2^0}$	$\times 2^{-1}$	$\times 2^{-2}$

ratio ($=x'/x$) (Figure 3a). The result shows a dramatic range, spanning from 2^{-128} to 2^{128} —with alternating bands based on polarity ($0 \rightarrow 1$ vs. $1 \rightarrow 0$). For example, when $e=64$, flipping $e[7]$ ($64 \rightarrow 192$) multiplies the value by $2^{128} \approx 3 \times 10^{38}$; conversely, flipping the same bit when $e=192$ ($\rightarrow 64$) yields $2^{-128} \approx 3 \times 10^{-39}$, effectively driving the value toward zero. Finally, flips that produce $e=0$ or $e=255$ enter BF16's special regimes (subnormals/zero or $\pm\infty/\text{NaN}$), causing discontinuities or NaNs (not shown).

Table I summarizes BF16's bit-level vulnerability. Exponent bits are highly fragile: their error ratios grow double-exponentially with bit index. Moreover, the effect is strongly polarity-dependent: $0 \rightarrow 1$ flips cause catastrophic amplification, whereas $1 \rightarrow 0$ flips induce large attenuation. The sign always introduces a ratio of -1 , but its impact is overshadowed by the extreme amplifications in exponent bits. In contrast, mantissa bits only introduce fractional changes, and their ratios decay exponentially with bit position, making them comparatively benign.

The final implication is that the *error magnitude* depends on both the scale factor and the original value ($|x' - x| = |SF - 1| \cdot |x|$). When $|x|$ is small, even large multiplicative factors can yield small absolute deviations; conversely, for large $|x|$, even small factors can produce large errors. Figure 3b visualizes this behavior for exponent-bit flips with $m=0$: the error scales with the original value and exhibits clear polarity asymmetry— $0 \rightarrow 1$ (amplification) dominates $1 \rightarrow 0$ (attenuation).

C. Memory Errors to DNN Accuracy

We next examine how these value errors translate into end-to-end accuracy loss using three DNN models: a CNN (ResNet-50) and two transformer-based LLMs (Llama-3.1-8B and Llama-3.2-1B), instantiated from Hugging Face [26] (PyTorch) with BF16 weights and activations. For ResNet-50, we report ImageNet-1k top-1 accuracy; for the LLMs, we report ARC-Easy accuracy (four-way multiple choice, so random guessing yields 0.25).

To simulate the effect of DRAM errors, we inject random bit flips in both the weights and intermediate activations of the networks during PyTorch-based inference (Section VI-B). For each target *Bit Error Rate* (BER), we inject errors uniformly across all bit positions of BF16 and assess the model’s accuracy. For each model and BER, we run 100 Monte-Carlo trials and summarize the resulting accuracies using box-and-whisker plots.

1) *Overall Robustness*: Figure 4(a-c) compare the overall robustness of the three models. ResNet is relatively tolerant to errors: its accuracy is essentially unchanged up to $\text{BER} = 10^{-9}$. At $\text{BER} = 10^{-8}$, the distribution broadens—most runs remain reasonably accurate while some collapse toward zero—indicating that only certain errors are catastrophic. By $\text{BER} = 10^{-7}$ most runs produce near-zero accuracy, and higher BERs completely destroy the model.

The two LLMs exhibit much higher vulnerability, losing accuracy at BERs roughly two to three orders of magnitude lower than the CNN. Llama-3.1 already shows noticeable degradation at $\text{BER} = 10^{-10}$, and all trials converge to chance-level performance by $\text{BER} = 10^{-9}$. Llama-3.2 is slightly more robust but follows the same trend: a non-trivial fraction of runs collapses at $\text{BER} = 10^{-10}$ and most do so by $\text{BER} = 10^{-9}$.

These results highlight two main observations:

- **LLMs can be significantly more fragile than CNNs**, likely due to token-based processing and transformer dynamics (attention amplification and residual pathways that can propagate outliers).
- **Accuracy varies widely across trials at a fixed BER**, suggesting that a small number of extreme value outliers trigger catastrophic degradation.

2) *Bit-level Robustness*: We now focus on *bit-level* robustness to validate the analysis presented in Section III-B. We limit error injection to a single bit position at a time, keeping all other bits error-free. Although we present results for Llama-3.2, similar trends are observed for other models as well.

Figure 4d illustrates these findings. Errors in sign and mantissa bits do not significantly affect accuracy until $\text{BER} = 10^{-5}$, while errors in exponent bits cause catastrophic failures even at much lower BERs, such as $\text{BER} = 10^{-12}$. This aligns with the analysis in Section III-B, where mantissa flips induce $\times(<1)$ perturbations, sign flips cause a bounded $\times(-1)$ change, and exponent flips lead to unbounded range changes.

Additionally, a few unlucky bit flips in critical exponent bits are sufficient to severely degrade LLM accuracy. Llama-3.2-

1B, with approximately 1B parameters and 156B activations, experiences ~ 1.5 flipped bits per trial at a $\text{BER} = 10^{-11}$ on $e[7]$. Despite this small error rate, 10 out of 100 trials drop to near random-guess accuracy. Even at $\text{BER} = 10^{-12}$, which corresponds to an average of 0.15 flipped bits per trial, 1 out of 100 trials still collapses, showing that a *single* unlucky exponent flip can be enough to break the model.

These bit-level results reinforce three key insights:

- **Not all bits are equally important**: Sign and mantissa bits have relatively minor effects, whereas exponent bits—especially high-order ones—dominate failures.
- **Few critical flips are enough to break the model**: A *single* unlucky exponent flip can be enough to destabilize the model’s performance, even with a low BER.
- **At increased BERs, protection must cover more than just the top exponent bit**: While $e[7]$ is the most vulnerable, other exponent bits also lead to severe accuracy degradation as the BER rises (e.g., 10^{-8} in the Llama-3.2 case).

Collectively, these observations show that DNNs, especially LLMs, are highly fragile to memory errors and require robust protection to maintain reliable performance.

IV. PRIOR WORK

Protecting DNN accuracy under memory errors has followed two broad paths. Industry predominantly deploys bit-precise ECC within a fixed redundancy budget, whereas academia increasingly explores semantics-aware protection, leveraging value distributions and model tolerance to extract more robustness per parity bit.

A. Industry Practice

Modern GPUs provision 16 bits of ECC per 256-bit block [27], [28]. This budget is typically allocated to either bit-level SEC–DED or *Single-Symbol Correction* (SSC) using 8-bit symbol RS codes. SEC–DED guarantees correction of any single-bit error and detection of any double-bit error within the 272-bit codeword (256 data + 16 parity). Conceptually, it designates *correction* for exactly the 272 single-bit error syndromes out of the $2^{16} = 65,536$ possible syndromes, and flags most other nonzero syndromes as detected uncorrectable errors (DUEs). This allocation yields very strong *detection* coverage (about 99.6% for > 2 -bit errors) and correspondingly low *silent data corruption* (SDC) risk.

Alternatively, vendors can encode 8-bit symbols with RS(34,32) to correct a *single* 8-bit symbol error within the block. Because more syndromes are devoted to correction events, residual *detection* coverage for uncorrectable multi-symbol faults is weaker (about 86.8%). In practice, vendors often prefer SEC–DED for its superior detection properties [29]–[31] and rely on higher-level mechanisms (e.g., checkpoint/rollback) to recover from DUEs. However, recent large-scale LLM training highlights that such recoveries incur substantial overhead, frequent job interruptions and degraded cluster availability [23].

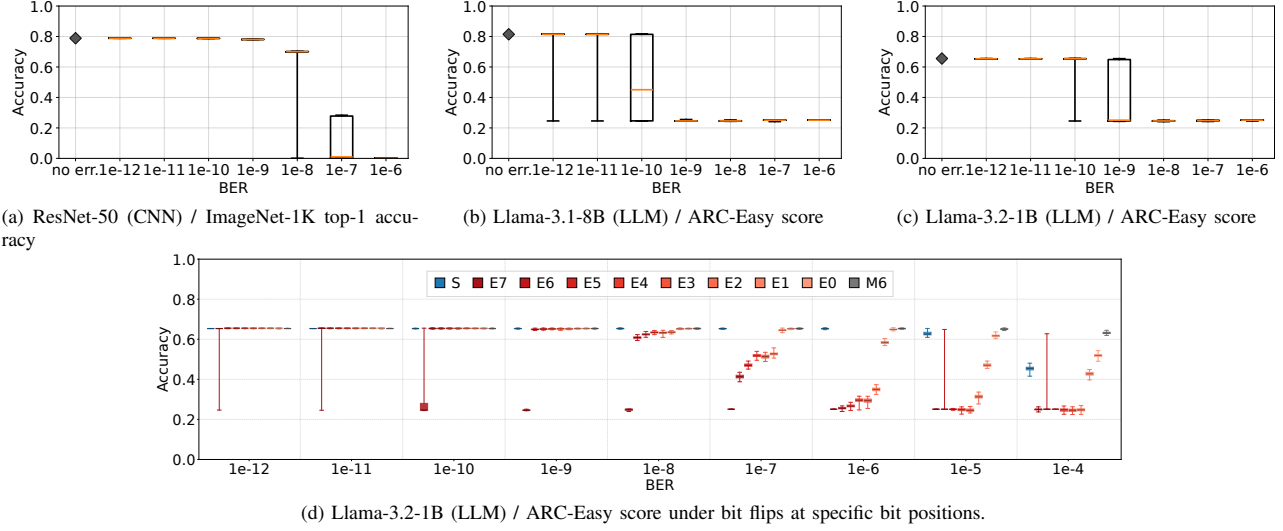


Figure 4. DNN accuracy under random bit flips across varying BERs.

B. Academic ECC Solutions

Recent research has proposed various techniques to mitigate the impact of memory errors on AI models. Unlike conventional ECC schemes, these methods often leverage the statistical redundancy and value distribution of DNN parameters to improve fault tolerance at low cost. Representative approaches include the following.

Variable Protection (VP) [32] exploits the inherent redundancy of neural network weights to enhance resilience against memory faults. By utilizing consistent bit patterns observed across weight groups, it enables lightweight error recovery with little impact on model accuracy. Although VP improves bit-level robustness, its protection coverage is limited by the characteristics of the model architecture and data representation.

Qin et al. [33] study the robustness of DNNs against storage-media errors and propose *Weight Nulling*, which adds a single parity bit per weight by repurposing the least significant bit. On a parity mismatch, the scheme replaces the corrupted weight with zero rather than correction. However, its parity check only detects odd-number bit errors and may lose important information when applied to weights with large magnitudes.

Value-Aware Parity Insertion (VAPI) [34] targets 8-bit quantized CNN weights. It observes that most weights lie near zero and adopts sign-magnitude representation so that some higher-order bits (e.g., b_6 , b_5) are rarely used. VAPI overwrites those “less important” bits with parity while keeping the overall storage overhead small. Using a DEC(64,50) code, it corrects up to two bit errors per 64-bit block of weights without retraining, but it still focuses on stored weights only and assumes 8-bit quantization with specific value distributions.

PoP-ECC [35] introduces a two-tier design that protects virtual parities (VPs) rather than raw weights, and stores only parities-of-parities (PPs) generated from the VPs. Combined with channel-wise quantization, this design provides strong

tolerance to multi-bit upsets. However, its focus on exact value recovery constrains how much efficiency it can ultimately achieve.

Despite their differences, these approaches share a common goal: protecting individual data bits. Some exploit intrinsic redundancy in neural parameters or alter data representations to achieve lightweight fault tolerance, often trading universality or coverage for storage efficiency.

C. DNN Reliability

Another line of research studies end-to-end reliability during DNN execution. Hong et al. [36] showed that hardware-induced errors can still severely affect DNN inference. To evaluate such vulnerabilities systematically, Mahmoud et al. [37] proposed a framework that quantifies the impact of hardware faults across multiple numerical formats. Most of these studies, however, focus on image-classification models.

Reliability analysis for LLMs has only recently emerged. Sun et al. [38] characterized the end-to-end impact of memory and compute errors during LLM inference and showed that error propagation in transformer architectures differs from that in image-classification models. Their study identifies vulnerable bit positions, but it does not provide a detailed characterization of the numerical mechanisms that drive this sensitivity.

Several mitigation techniques aim to improve the fault resilience of DNNs. Sullivan et al. [39] proposed *SwapCodes*, a hardware–software cooperative mechanism that leverages register-file ECC and instruction duplication to detect transient errors in GPU pipelines. Wasim et al. [40] showed that initializing the classification layer with text-guided embeddings can improve the resilience of image classifiers by increasing classification margins and reducing sensitivity to activation faults. Chen et al. [41] proposed *Ranger*, which constrains layer activations to statistically profiled ranges and suppresses catastrophic fault-induced value spikes.

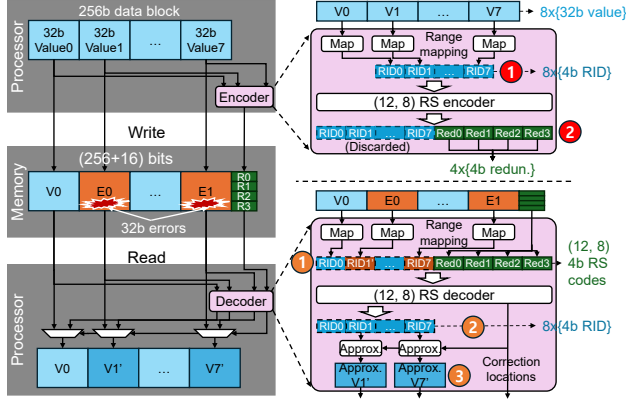


Figure 5. RangeGuard on a 256-bit block containing eight FP32 values.

V. RANGE GUARD

This section presents RangeGuard, a novel ECC framework that preserves DNN accuracy under severe memory errors. With the standard HBM ECC budget of 16 parity bits per 256-bit block (6.25% redundancy), RangeGuard tolerates more than 64 flipped data bits per block—roughly $8\times$ the raw-bit coverage of conventional bit-centric schemes, which can correct up to 8 flipped bits under the same budget.

Conventional ECC protects individual *data bits*, so redundancy grows with the number of bits being guarded. Some prior designs attempt to save redundancy by protecting only “important” bits (e.g., integer MSBs). For floating-point DNNs, however, this approach is fragile: as shown in Section III-B, flips in sign or exponent can amplify error magnitude beyond the original value, and even the most-significant mantissa bit shifts a value by one-half. Simply prioritizing a subset of bits does not align with what actually drives model failure.

RangeGuard instead protects compact, per-value *metadata* in the form of *Range Identifiers (RIDs)*. The numeric domain is partitioned into a small number of ranges, and each value is mapped to the index of its range (e.g., a 4-bit RID). Because RIDs are much smaller than the underlying values, they can be strongly protected within a tight parity budget. On a fault, RangeGuard first corrects the RID and then *approximately* reconstructs the value by substituting a representative of that range, bounding the post-repair error by the range width.

A. Architecture

Figure 5 shows RangeGuard applied to a 256-bit block with eight FP32 values. In this example, two FP32 values are completely corrupted (Value1→Error0 and Value7→Error1, 64 bits in total).

1) *Write path (encode)*: Each FP32 value is first mapped to one of 16 predefined ranges (1), producing a 4-bit RID. The eight RIDs are then encoded using a RS code over 4-bit symbols, RS(12, 8), which generates four 4-bit parity symbols (16 bits total) (2). The encoder discards the explicit RIDs and stores only the original 256-bit data plus the 16-bit parity.

2) *Memory errors*: During storage or transfer, memory faults can perturb the data. We categorize them as:

- **Intra-range errors**: Bit flips change the value but keep it within the same range (e.g., lower mantissa flips). These are considered semantically benign.
- **Inter-range errors**: Bit flips move the value into a different range (e.g., sign/exponent flips or large mantissa shifts) and are considered harmful.

3) *Read path (decode)*: On a read, RangeGuard regenerates the eight RIDs from the received FP32 values (1); corrupted data may yield erroneous RIDs. The regenerated RIDs and the stored RS(12, 8) parity are passed to the RS decoder, which corrects up to two erroneous RID symbols (2). For each corrected RID, the corresponding FP32 value is replaced with the representative of that range (3), achieving *bounded approximate correction*.

If the RS code can correct t symbols, RangeGuard tolerates up to t *inter-range* errors (RID errors) per block, while any number of benign *intra-range* errors pass through without consuming correction capacity. Each value position uses a small mux: if the RID decoder flags no error (either a true no-error case or an intra-range error), the raw FP32 value is forwarded; if an inter-range error is corrected, the range representative is used instead. When the number of inter-range errors exceeds t , the decoder raises a *DUE* (or results in SDC in rare cases).

Conceptually, RIDs resemble quantization levels, but with a crucial difference: RangeGuard uses ranges only as *recovery targets*, not as the primary storage format. Error-free values are stored and used at full precision; only corrupted values are snapped back to their range representatives. This metadata-centric, range-aware design is what allows RangeGuard to provide strong, bounded-error protection under a fixed GPU ECC budget.

B. Range Mapping

Range mapping decides (i) how values are grouped into ranges and (ii) how large the post-repair error can be, so it has a direct impact on DNN accuracy. If ranges are too *wide*, RangeGuard wastes its limited RID space on rarely used magnitudes (e.g., 2^{127}) and cannot describe common values precisely. If ranges are too *narrow*, many RIDs are spent on almost identical values (e.g., $[2^{-126}, 2^{-125})$ vs. $[2^{-125}, 2^{-124})$) for only marginal MAE improvement. The goal is to construct a *RangeMap* that balances coverage of realistic value distributions against a tight RID budget and bounded-error requirements.

1) *Ideal Mapping*: We define the RangeMap objective as minimizing *Mean Absolute Error (MAE)* after repair:

$$\text{MAE} = \int_{-\infty}^{\infty} \text{PDF}(x) |x - \hat{x}(x)| dx, \quad (4)$$

where $\text{PDF}(\cdot)$ is the distribution of x , and $\hat{x}(x)$ is the representative of the range that x falls into.

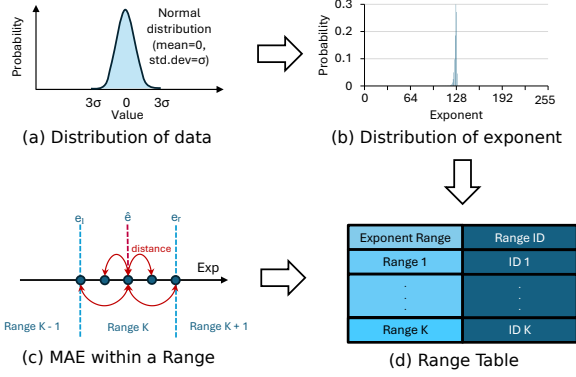


Figure 6. Construction algorithm for simple range mapping.

If x follows a normal distribution with mean 0 and standard deviation σ , i.e., $x \sim \mathcal{N}(0, \sigma^2)$, and we use 2-bit RIDs (4 ranges) for BF16, we parameterize the four ranges as

$$(-\infty, c_1\sigma), [c_1\sigma, c_2\sigma), [c_2\sigma, c_3\sigma), [c_3\sigma, +\infty),$$

with corresponding representatives $r_1\sigma, r_2\sigma, r_3\sigma, r_4\sigma$.

For a given choice of $\{c_i, r_i\}$, the MAE is

$$\text{MAE} = \sum_{i=1}^4 \int_{c_{i-1}\sigma}^{c_i\sigma} \text{PDF}(x) |x - r_i\sigma| dx, \quad (5)$$

where $c_0 = -\infty$ and $c_4 = +\infty$.

We solve this optimization once offline using standard L1-optimal scalar quantization. For the normalized Gaussian, the optimal thresholds and representatives are

$$(c_1, c_2, c_3) = (-0.8217, 0, 0.8217)$$

$$(r_1, r_2, r_3, r_4) = (-1.2657, -0.3778, 0.3778, 1.2657).$$

These normalized values can be reused for any format (e.g., FP32, BF16, INT8) whose data follow a similar distribution after scaling by σ .

2) *Simple Mapping*: In practice, RangeGuard may apply several RangeMaps in parallel within a block and support multiple formats at once. To keep hardware simple, we restrict the mapping logic to the exponent field (motivated by Section III-B). For BF16, the construction procedure is summarized in Figure 6.

We first derive the exponent distribution for the same distribution of a zero-mean Gaussian with standard deviation σ . Let e denote the 8-bit BF16 exponent. Its probability mass function is

$$P(e = k) = 2 \left[\Phi\left(\frac{2^{k+1}-127}{\sigma}\right) - \Phi\left(\frac{2^k-127}{\sigma}\right) \right], \quad (6)$$

where $\Phi(\cdot)$ is the Cumulative Distribution Function (CDF) of the standard normal distribution.

Next, we build a range table over the exponent domain that minimizes MAE at the *magnitude* level. Let the exponent space be

$$\mathcal{E} = \{0, 1, \dots, 255\},$$

Table II
EXAMPLE OF A 4-ENTRY RANGE TABLE WHEN $\sigma = 4$.

Exponent Range	Value Range	Boundary Exponent	Representative Value
[0, 127]	(0, 2)	01111111	0.5
128	[2, 4)	10000000	2
129	[4, 8)	10000001	4
[130, 255]	[8, ∞)	11111111	8

and define a monotonic mapping function representing the numerical scale of each exponent:

$$f(e) = 2^{e-127}.$$

We partition $\mathcal{E}$ into K contiguous intervals $\{[l_k, r_k]\}_{k=1}^K$, and assign a representative exponent $\hat{e}_k \in [l_k, r_k]$ to each interval. In this case, all values within a given interval are bounded by the same representative exponent after RangeGuard recovery. Therefore, the overall MAE can be expressed as the weighted sum of reconstruction errors within each range:

$$L = \sum_{k=1}^K \sum_{e=l_k}^{r_k} P(e) |f(e) - f(\hat{e}_k)|.$$

Through this process, we obtain a *globally optimal range table* that minimizes the overall MAE under the given exponent distribution and average error model.

Table II shows an example 4-entry range table for $\sigma = 4$. This σ is conservatively chosen so that most LLM values (roughly $\pm 3\sigma \approx \pm 12$) fall within the covered region.

C. ECC Configurations

RangeGuard is a flexible framework that can be tuned to different reliability targets under a fixed 16-bit parity budget. We consider two RS-based designs that trade off correction strength and reconstruction fidelity: *8b Single-Symbol Correction (SSC)* and *4b Double-Symbol Correction (DSC)*, shown in Figure 7.

In both cases, protection is organized at 32-bit granularity: each 32-bit value corresponds to one RS symbol, so common burst faults (e.g., 16-bit local wordline faults or 32-bit subwordline driver faults) fall within a single symbol and can be corrected with less redundancy.

1) *8b SSC*: The 8b SSC configuration uses 8-bit symbols and an RS(10,8) code, where 8 data symbols and 2 parity symbols form one ECC word. A 256-bit block is split into eight 32-bit values, each mapped to an 8-bit RID symbol; the 16-bit ECC lane stores two parity symbols and enables correction of any single symbol in the block (up to 32 corrupted data bits). This configuration provides fine-grain range information and thus higher-quality approximate reconstruction.

2) *4b DSC*: The 4b DSC configuration uses 4-bit symbols with an RS(12,8) code, comprising 8 data symbols and 4 parity symbols. Each 32-bit value is mapped to a 4-bit RID symbol; the 16-bit ECC lane holds four parity symbols and supports correction of up to two symbols (up to 64 corrupted data bits). Because each value gets fewer RID bits, the recovered

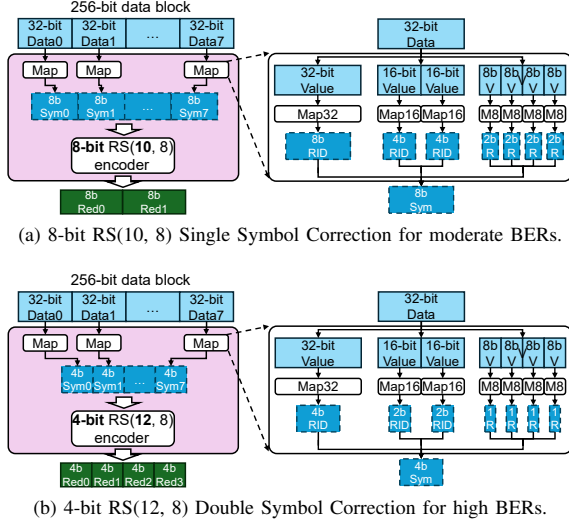


Figure 7. RangeGuard ECC configurations.

values are coarser, but the configuration offers stronger protection against overlapping faults. The 12-symbol word length respects the RS maximum of 15 symbols for 4-bit symbols.

D. Multi-Format Support

Modern DNNs employ multiple numeric formats (e.g., FP32, BF16, FP8, INT8) and exhibit distinct value distributions across layers (weights vs. activations, early vs. late blocks). RangeGuard adapts to this diversity in a structured way. First, RangeGuard can host multiple RangeMaps in parallel, each tuned to a specific format and tensor class (e.g., FP32 weights, BF16 activations, FP8 logits). As shown in Figures 7a and 7b, each RangeMap instance includes sub-maps for 32-, 16-, and 8-bit values. For sub-32b values, each sub-map produces smaller RIDs, and several of these are packed into a single *RID symbol* to match the ECC symbol width. As the value size shrinks, more values fit in 32-bit data, so we increase the number of sub-maps and reduce RID bits per value. This allows the same ECC budget to be reused efficiently across formats without assigning ranges to values that never appear. On the downside, this incurs more hardware costs by requiring multiple instances of sub-maps per 32-bit data.

We choose the per-value RID widths to satisfy both semantic fidelity and DRAM fault characteristics. In particular, common DRAM fault modes can corrupt up to 32 aligned bits within a block (Section III-A). To make such a fault appear as a single symbol error, RangeGuard packs the RIDs of values that share the same 32-bit-aligned region into one RID symbol. A 32-bit region can contain two 16-bit values, four 8-bit values, or eight 4-bit values. For example, using 4/2/1-bit RIDs for 16/8/4-bit values lets all RIDs in a 32-bit region fit into one 8-bit symbol, enabling correction with a single 8-bit-symbol SSC. Alternatively, using 2/1-bit RIDs for 16/8-bit values packs the region into one 4-bit symbol,

enabling correction with a single 4-bit-symbol code while leaving additional correction capability for multiple faults.

Second, the memory controller selects the appropriate sub-map using a small tag embedded in the physical address, similar to prior address-based ECC selection schemes [6]. Specifically, RangeGuard repurposes unused upper address bits (e.g., bit[57:54]) as a *Map Tag*, enabling the controller to identify the target sub-map without extra pins or sideband signals and to switch mappings on the fly. On reads, the decoder consults this tag to pick the correct sub-map and reconstruct the representative value from the decoded RID. For data regions that require exact recovery, the Map Tag can also select a conventional ECC scheme (e.g., SEC-DED), thereby providing exact but weaker protection. This mechanism allows RangeGuard to support different data types and protection requirements with minimal hardware changes.

Taken together, these components—RID-based architecture, SSC/DSC ECC configurations, exponent-aware RangeMaps, and multi-format support—turn RangeGuard into a drop-in, GPU-compatible protection layer that spends the ECC budget on what actually matters: keeping values in safe numeric ranges. The memory system still stores and moves raw data at full precision, but when faults occur, RangeGuard detects harmful range changes, snaps corrupted values back to well-chosen representatives, and lets benign noise pass through. In the next section, we show that this design yields strong coverage under realistic DRAM fault modes, preserves CNN and LLM accuracy at high BERs, and incurs negligible area and performance overhead.

VI. EVALUATION

This section evaluates the reliability benefits and hardware costs of RangeGuard compared to existing state-of-the-art error correction schemes.

A. Error Coverage

We begin by comparing the correction capabilities of RangeGuard against common fault modes encountered in DRAM systems. These fault modes were derived from prior DRAM error studies [5], [42] and include:

- **Single-Error (SE) Fault:** A fault that causes a single bit to flip within a block, typically resulting from faulty cells or marginal bitlines.
- **Double Adjacent Error (DAE) Fault:** A fault that induces two adjacent bit errors within a block, often occurring due to cell-to-cell bridging faults or TSV faults.
- **16-bit Error (16E) Fault:** A fault that causes up to 16 bit errors within a 16-bit boundary, which can result from issues like local wordline or column select line faults.
- **32-bit Error (32E) Fault:** A fault that leads to up to 32 bit errors within a 32-bit boundary, typically caused by sub-wordline driver faults.
- **Full-Chip (FC) Fault:** A fault that corrupts all bits within a block.

Because RangeGuard can tolerate a single instance of most fault modes except FC, we evaluate both single-fault and

Table III
A COMPARISON OF CORRECTION AND DETECTION COVERAGE IN VARIOUS FAULT SCENARIOS.

Fault Modes	Result	Baseline	Weight Nulling	VAPI	RG 8b SSC	RG 4b DSC
SE	CE DUE	100.000 0.000	0.000 100.000	100.000 0.000	BE 100.000	BE 100.000
DAE	CE DUE SDC	100.000 0.000 0.000	0.000 74.997 25.003	100.000 0.000 0.000		
16E	CE DUE SDC	100.000 0.000 0.000	0.000 50.011 49.989	0.209 99.782 9×10^{-3}		
32E	CE DUE SDC	3×10^{-3} 99.996 1×10^{-3}	0.000 25.007 74.993	9×10^{-6} 99.994 6×10^{-3}		
SE+SE	CE BE DUE SDC	4.949 95.051 0.000	0.000 94.117 5.883	100.000 0.000 0.000		
SE+DAE	CE BE DUE SDC	4.633 95.367 0.000	0.000 72.245 27.755	93.899 6.101 0.000		
SE+16E	CE BE DUE SDC	0.000 99.999 1×10^{-3}	0.000 50.001 49.999	0.173 99.826 1×10^{-3}		
SE+32E	CE BE DUE SDC	0.000 99.999 2×10^{-3}	0.000 25.003 74.997	1×10^{-5} 99.999 8×10^{-4}		
FC	CE BE DUE SDC	0.000 99.998 2×10^{-3}	0.000 2×10^{-3} 99.998	0.000 100.000 0.000		

double-fault cases within a block. For each fault scenario, we randomly select the fault location(s) within a block and inject bit corruptions within the affected region. Except for SE, each bit within the fault boundary is flipped independently with 50% probability. We then apply the ECC decoder and classify each outcome as a *Corrected Error (CE)*, *DUE*, or *SDC*. For RangeGuard, we report *Bounded Error (BE)* instead of CE because the design targets bounded approximate recovery rather than exact bitwise correction. We repeat each experiment 10^9 times and report the resulting outcome probabilities.

We compare two configurations of RangeGuard (8-bit SSC and 4-bit DSC) against a baseline, Weight Nulling [33], and VAPI [34]. Our baseline models an HBM3-style stack that uses a 16-bit RS(19,17) code as O-ECC to correct storage-side faults inside the device and a 16-bit cyclic redundancy check (CRC16) as S-ECC to provide end-to-end error detection. We assume all errors occur during storage to go through O-ECC and S-ECC. For Weight Nulling and VAPI, which repurpose some unused exponent bits for parity, we assume that the value distribution allows for this repurposing.

Table III compares error coverage across the schemes. The baseline provides exact correction only when faults remain within the O-ECC correction range. Its CE coverage is already negligible for 32E, correcting only $3 \times 10^{-3}\%$ of cases, and it completely loses CE capability once the fault pattern exceeds the O-ECC range, as in SE+16E and SE+32E.

Weight Nulling uses parity bits only for detection and offers no correction; its parity-based checks miss most multi-bit patterns. VAPI corrects SE, SE+SE, and DAE well, but its

Table IV
FAULT-MODE DISTRIBUTION AND EXPECTED FLIPPED BITS USED DURING THE DNN ACCURACY MEASUREMENT.

Fault Mode	Fault Ratio (A)	Error Count (B)	Error Ratio (A×B)
SE	$0.009 \times BER$	1	$0.009 \times BER$
DAE	$0.012 \times BER$	2	$0.023 \times BER$
16E	$0.022 \times BER$	8	$0.175 \times BER$
32E	$0.050 \times BER$	16	$0.793 \times BER$
Total			BER

bit-level protection is still weak against wider multi-bit faults.

In contrast, RangeGuard provides the strongest overall coverage. RG 8b SSC corrects all single faults (except FC), while RG 4b DSC extends support to two simultaneous faults. Both configurations show negligible misdetection, with an FC misdetection rate of $2 \times 10^{-3}\%$. Thus, RangeGuard is robust for both common (single-fault) and rare (two-fault) scenarios; faults that trigger more than three range changes are expected to be exceedingly rare.

B. DNN Accuracy

We next compare DNN accuracy across schemes under more realistic DRAM fault patterns. Unlike the single-bit injection experiments in Section III-C, here we randomly inject SE, DAE, 16E, and 32E faults to reflect common post-on-die-ECC failures. We adopt fault ratios from a DDR5 fault classification study [42] and weight each mode by its expected number of flipped bits so that the combined contribution of all modes matches the target BER (Table IV). This setup lets us stress each scheme under a more realistic mix of sparse and bursty multi-bit errors rather than idealized single-bit flips.

During PyTorch-based inference, we inject randomly generated faults into stored weights and intermediate activations before subsequent layers consume them, following the methodology of recent DNN fault-injection frameworks [38], [43]–[46]. For each fault, we flip bits within the affected region with 50% probability, apply the corresponding ECC algorithm to the corrupted block, and update the tensor only when the outcome is uncorrectable or bounded-corrected. We then measure the final inference accuracy and repeat the experiment 100 times for each BER and protection scheme. For the LLM experiments, we conduct inference with lm-evaluation-harness [47].

Figure 8 shows how accuracy degrades as BER increases. The *no-protection* case behaves similarly to the single-bit experiments: ResNet-50, Llama-3.1, and Llama-3.2 exhibit sharp accuracy drops around BERs of 10^{-8} , 10^{-10} , and 10^{-10} , respectively, and quickly approach chance-level performance beyond those points. For the LLMs, the run-to-run spread is noticeably worse than in the single-bit case, likely because clustered faults (16E, 32E) more frequently strike exponent fields and generate extreme outliers that dominate attention and residual paths.

Weight Nulling and VAPI provide only limited improvement over this baseline. Their weak coverage for bursty multi-bit

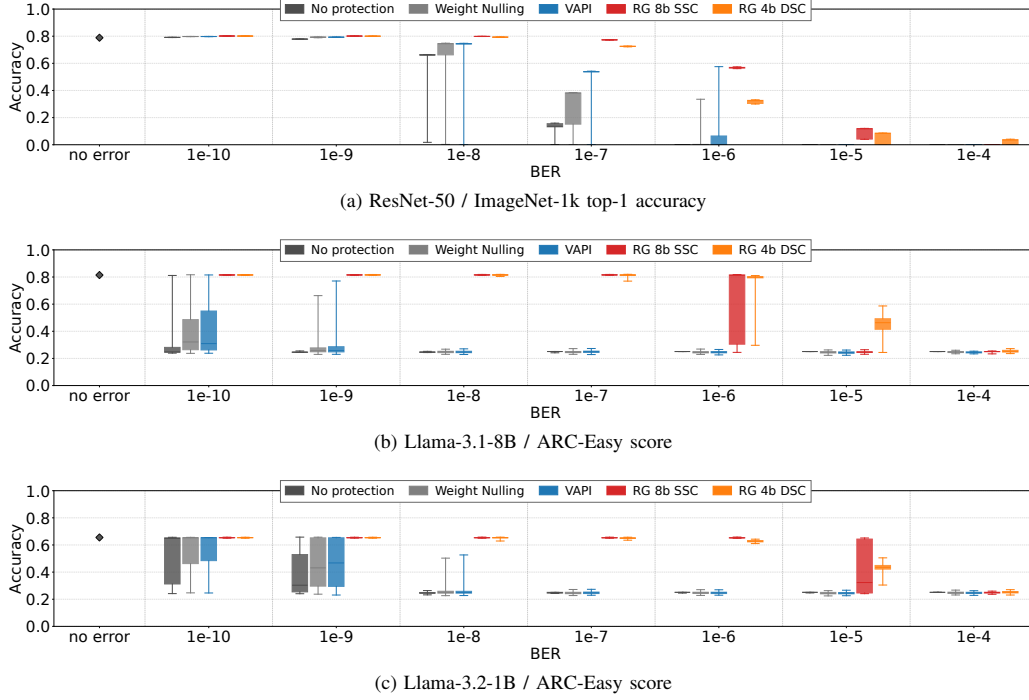


Figure 8. DNN accuracy degradation with varying BERs.

errors leaves substantial accuracy loss at moderate BERs, with curves that remain close to the no-protection case, especially for LLMs.

In contrast, RangeGuard closely tracks the no-error baseline across a wide BER range despite the presence of multi-bit errors. ResNet and Llama-3.1 maintain near-baseline accuracy up to $\text{BER} = 10^{-7}$, and Llama-3.2 remains robust up to $\text{BER} = 10^{-6}$, significantly extending the usable BER envelope compared to other schemes. Even beyond these thresholds, RangeGuard degrades much more gracefully than bit-centric approaches, confirming that bounding range changes rather than raw bit mismatches is an effective strategy for preserving DNN and LLM accuracy under realistic fault patterns. Between the two RangeGuard configurations, the better choice is model-dependent: 8b SSC outperforms 4b DSC on ResNet and Llama-3.2 by providing more accurate approximate correction with its 16-entry RangeMap, while 4b DSC is preferable for Llama-3.1 thanks to its stronger tolerance to overlapping faults and higher symbol-level correction capability.

C. Hardware Overheads

To estimate the hardware cost of RangeGuard, we implement its encoder and decoder in SystemVerilog and evaluate two configurations. For RG 8b SSC, the encoder extracts 8-bit RIDs using a flip-flop-based RangeMap. For each 32B access, it compares the 8-bit exponent of each of the sixteen 16-bit values against all 16 RangeMap entries in parallel, requiring 256 8-bit comparators but completing RID extraction in one cycle. A second cycle generates the redundancy bits. RG 4b DSC uses the same structure with a smaller 4-entry

Table V
HARDWARE OVERHEADS OF BF16 RANGEGUARD

Metric	Component	RG 8b SSC	RG 4b DSC
Area	Encoder (total)	3,300 μm^2 (6,500)	800 μm^2 (1,600)
	- RangeMaps	3,100 μm^2 (6,200)	650 μm^2 (1,300)
	- ECC encoder	200 μm^2 (300)	150 μm^2 (300)
	Decoder (total)	7,800 μm^2 (15,400)	4,200 μm^2 (8,200)
Power	- RangeMaps	6,700 μm^2 (13,200)	1,600 μm^2 (3,000)
	- ECC decoder	1,100 μm^2 (2,200)	2,600 μm^2 (5,200)
	Decoder (total)	0.54 mW	0.18 mW
	Encoder (total)	0.61 mW	0.63 mW

RangeMap, reducing the lookup to 64 8-bit comparators, and uses a different generator matrix for redundancy generation.

The decoder regenerates RIDs from the received data in one cycle using the same RangeMap-based lookup structure as the encoder. For area reporting, we model a separate RangeMap in the decoder, although the encoder and decoder can share a single flip-flop-based table in practice. In the next cycle, the decoder computes the syndrome and performs error detection. When an error is detected, it applies correction using a modified Berlekamp–Massey algorithm hardened in logic gates [48]–[50], requiring one additional cycle per correction.

We synthesize the RTL using Synopsys Design Compiler with a UMC 28nm standard-cell library at 1GHz. We report area in NAND2 equivalents to make the results more process independent, and we further break down the area into RangeMap and ECC logic to help estimate the overhead of configurations with multiple RangeMaps.

Table V presents the results. The combined area of the

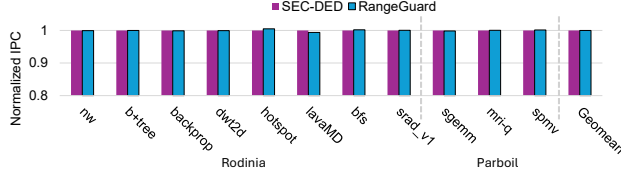


Figure 9. IPC comparison between SEC-DED and RangeGuard.

8-bit SSC encoder and decoder is $11,100 \mu\text{m}^2$, which is larger than that of the 4-bit DSC configuration ($5,000 \mu\text{m}^2$) due to its larger RangeMap. Nevertheless, this total overhead corresponds to only 21,900 NAND2 equivalents (87,600 transistors). Considering modern GPUs with billions of transistors this is a small portion of the die area (e.g., $5\text{e-}7$ in 208B-transistor Blackwell).

D. GPU System Performance

To assess the performance cost of the protection, we use the cycle-level GPU simulator Accel-Sim [51], configured to model an NVIDIA V100 GPU. Since memory errors are rare (occurring on the scale of days or months), we focus on evaluating RangeGuard’s impact in error-free scenarios.

We adjust the read latency to reflect the decoding overhead. We assume that RangeGuard requires 2 cycles to return error-free data (immediately after error detection stage), while the baseline already spends 1 cycle on error detection, resulting in a net 1-cycle increase in read latency. We do not increase the write latency, because the baseline already uses 1 cycle for encoding and the additional cycle can be hidden while the request waits in the memory-controller queue or during DRAM write latency (i.e., the interval between the DRAM write command and the arrival of write data). We evaluate 11 workloads from Rodinia [52] and Parboil [53].

Figure 9 shows the results. Owing to the GPU’s throughput-oriented design, the extra cycle of read latency has a negligible effect on performance, reducing geomean Instructions-Per-Cycle (IPC) by only 0.008%. Even for the most memory-intensive workloads in our suite, the IPC drop remains below 0.05%, while compute-bound workloads are virtually unaffected. These results indicate that RangeGuard’s protection can be deployed in GPU systems without sacrificing throughput and system performance.

E. Sensitivity Analysis

To study how semantic protection in RangeGuard depends on mapping parameters, we conduct a sensitivity analysis of range mapping by varying the sigma value and mapping granularity, using the same experimental methodology as in Section VI-B.

Sigma Size: To evaluate the sensitivity of the σ value, we first constructed a RangeMap using the simple mapping scheme described in Section V-B while using 2-bit RIDs. As discussed in Section V-B, σ plays a critical role in balancing parameter coverage against mapping resolution. Fig. 10a highlights that protection efficacy is highly dependent on an

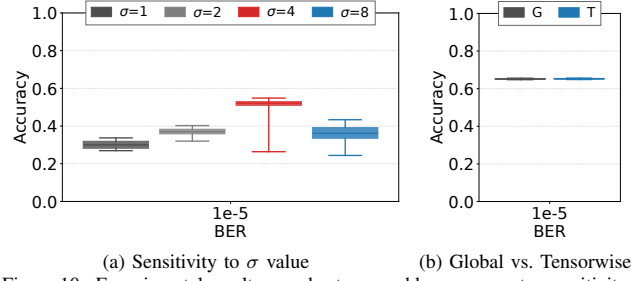


Figure 10. Experimental results on robustness and hyperparameter sensitivity.

optimal σ . Both excessively small and large σ values fail to adequately capture the value distribution, thereby undermining the semantic protection and triggering a sharp decline in model accuracy.

Mapping Granularity: We compared global mapping (G) and tensorwise mapping (T) to evaluate the influence of mapping granularity on diverse internal distributions (Fig. 10b). Global mapping uses a fixed configuration ($\sigma = 4$, 4-bit RID), and tensorwise mapping generates dedicated RangeMaps via the Lloyd-Max algorithm [54], [55] modified to minimize MAE. While this approach provides more precise reconstruction, it incurs substantial area overhead in the encoder and decoder to manage the massive number of unique RangeMaps. Despite its simplicity, global mapping provides sufficient resilience. This is because, at a bit error rate of 10^{-5} , the precision of the global mapping already ensures the semantic integrity of the model without requiring tensor-specific optimization.

VII. CONCLUSION

As DRAM scales and 3D-stacked memories become pervasive, multi-bit faults are increasingly common and especially harmful for modern DNNs and LLMs. We show that a few exponent flips can create extreme outliers, collapsing accuracy at very low BERs where prior bit-centric mitigation schemes already break down.

This paper presented *RangeGuard*, a metadata-centric framework that protects *ranges* instead of raw bits. RangeGuard encodes compact Range Identifiers (RIDs) into the existing 16-bit parity bits per 32-byte block, detects *inter-range* deviations, and performs bounded approximate correction by restoring the range and substituting a representative value. We develop MAE-aware RangeMap construction and two concrete configurations (8b SSC, 4b DSC) that fit within current GPU memory interfaces. Under realistic DRAM fault modes, RangeGuard tolerates errors affecting 64+ data bits per block while keeping errors bounded, preserves near-baseline accuracy for CNNs and LLMs at high BERs, and incurs only a negligible performance impact (0.008% IPC loss). These results demonstrate that reallocating limited redundancy from bit correctness to semantic range consistency is a practical and effective direction for building fault-resilient DNN/LLM systems on future error-prone memories.

ACKNOWLEDGMENT

This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2026-25519569, Development of an Operating Interface and Platform for LPDDR6-PIM, 20%; No. RS-2026-25520910, Development of LPDDR6 Interface, Low-power and High-speed Memory Network, 20%; No. RS-2019-II190421, AI Graduate School Support Program, Sungkyunkwan University, 20%; No. RS-2023-00228970, Development of Flexible SW-HW Conjunctive Solution for On-edge Self-supervised Learning, 20%; No. RS-2025-10692981, AI Semiconductor Innovation Research Center, 20%).

REFERENCES

- [1] JEDEC Solid State Technology Association, "JESD79-4D: DDR4 SDRAM Standard," JEDEC, Standard, 2021.
- [2] JEDEC Solid State Technology Association, "JESD79-5C.01: DDR5 SDRAM Standard," JEDEC, Standard, 2024.
- [3] C. Li, Y. Zhang, J. Wang, H. Chen, X. Liu, T. Huang, L. Peng, S. Zhou, L. Wang, and S. Ge, "From correctable memory errors to uncorrectable memory errors: What error bits tell," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.
- [4] R. Wu, S. Zhou, J. Lu, Z. Shen, Z. Xu, J. Shu, K. Yang, F. Lin, and Y. Zhang, "Removing obstacles before breaking through the memory wall: A close look at HBM errors in the field," in *USENIX Annual Technical Conference (ATC)*, 2024.
- [5] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S. H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, "A 16 GB 1024 GB/s HBM3 DRAM With Source-Synchronized Bus Design and On-Die Error Control Scheme for Enhanced RAS Features," *IEEE Journal of Solid-State Circuits (JSSC)*, 2023.
- [6] D. H. Yoon and M. Erez, "Virtualized and flexible ECC for main memory," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2010.
- [7] A. N. Udipi, N. Muralimanohar, R. Balasubramonian, A. Davis, and N. P. Jouppi, "LOT-ECC: Localized and tiered reliability mechanisms for commodity memory systems," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2012.
- [8] J. Kim, M. Sullivan, and M. Erez, "Bamboo ECC: Strong, safe, and flexible codes for reliable computer memory," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2015.
- [9] J. Kim, M. Sullivan, S. Lym, and M. Erez, "All-Inclusive ECC: Thorough end-to-end protection for reliable computer memory," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2016.
- [10] D. Kim, J. Lee, W. Jung, M. B. Sullivan, and J. Kim, "Unity ECC: Unified memory protection against bit and chip errors," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.
- [11] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, 1960.
- [12] H. Jun, J. Cho, K. Lee, H.-Y. Son, K. Kim, H. Jin, and K. Kim, "HBM (High Bandwidth Memory) DRAM Technology and Architecture," in *IEEE International Memory Workshop (IMW)*, 2017.
- [13] S. Ha, S. Lee, G. Bae, D. Lee, S. Kim, B. Woo, N. Lee, Y. Lee, and S. Pae, "Reliability Characterization of HBM featuring HK+MG Logic Chip with Multi-stacked DRAMs," in *IEEE International Reliability Physics Symposium (IRPS)*, 2023.
- [14] L. Li, P. Ton, M. Nagar, and P. Chia, "Reliability Challenges in 2.5D and 3D IC Integration," in *IEEE Electronic Components and Technology Conference (ECTC)*, 2017.
- [15] K. C. Chun, Y. K. Kim, Y. Ryu, J. Park, C. S. Oh, Y. Y. Byun, S. Y. Kim, D. H. Shin, J. G. Lee, B.-K. Ho, M.-S. Park, S.-J. Cho, S. Woo, B. M. Moon, B. Kil, S. Ahn, J. H. Lee, S. Y. Kim, S.-K. Choi, J.-S. Jeong, S.-G. Ahn, J. Kim, J. J. Kong, K. Sohn, N. S. Kim, and J.-B. Lee, "A 16-GB 640-GB/s HBM2E DRAM with a data-bus window extension technique and a synergetic on-die ECC scheme," *IEEE Journal of Solid-State Circuits (JSSC)*, 2020.
- [16] H. Jun, S. Nam, H. Jin, J.-C. Lee, Y. J. Park, and J. J. Lee, "High-bandwidth memory (HBM) test challenges and solutions," *IEEE Design & Test*, 2016.
- [17] J. Lee, K. Cho, C. K. Lee, Y. Lee, J.-H. Park, S.-H. Oh, Y. Ju, C. Jeong, H. S. Cho, J. Lee, T.-S. Yun, J. H. Cho, S. Oh, J. Moon, Y.-J. Park, H.-S. Choi, I.-K. Kim, S. M. Yang, S.-Y. Kim, J. Jang, J. Kim, S.-H. Lee, Y. Jeon, J. Park, T.-K. Kim, D. Ka, S. Oh, J. Kim, J. Jeon, S. Kim, K. T. Kim, T. Kim, H. Yang, D. Yang, M. Lee, H. Song, D. Jang, J. Shin, H. Kim, C. Baek, H. Jeong, J. Yoon, S.-K. Lim, K. Y. Lee, Y. J. Koo, M.-J. Park, J. Cho, and J. Kim, "13.4 A 48GB 16-High 1280GB/s HBM3E DRAM with All-Around Power TSV and a 6-Phase RDQS Scheme for TSV Area Optimization," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2024.
- [18] A. Farmahini-Farahani, S. Gurumurthi, G. Loh, and M. Ignatowski, "Challenges of High-Capacity DRAM Stacks and Potential Directions," in *Workshop on Memory Centric High Performance Computing*, 2018.
- [19] C. Y. Lee, S. Kim, H. Jun, K. W. Kim, and S. J. Hong, "TSV technology and challenges for 3D stacked DRAM," in *Symposium on VLSI Technology (VLSI-Technology): Digest of Technical Papers*, 2014.
- [20] T. Kim, J. Lee, Y. Kim, H. Park, H. Hwang, J. Kim, H. Jung, and D. W. Kim, "Thermal Improvement of HBM with Joint Thermal Resistance Reduction for Scaling 12 Stacks and Beyond," in *IEEE Electronic Components and Technology Conference (ECTC)*, 2023.
- [21] M. V. Beigi, Y. Cao, S. Gurumurthi, C. Recchia, A. Walton, and V. Sridharan, "A Systematic Study of DDR4 DRAM Faults in the Field," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
- [22] X. Du and C. Li, "Predicting uncorrectable memory errors from the correctable error history: No free predictors in the field," in *International Symposium on Memory Systems (MEMSYS)*, 2021.
- [23] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Srivankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Young, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raptarthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collet, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gouget, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang,

- Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Gostand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Sweet, G. Halpern, G. Herman, G. Sizov, G. J. Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Y. S. Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosenbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.
- [24] S. Gurumurthi, K. Lee, M. Jang, V. Sridharan, A. Nygren, Y. Ryu, K. Sohn, T. Kim, and H. Chung, "HBM3 RAS: Enhancing resilience at scale," *IEEE Computer Architecture Letters (CAL)*, 2021.
- [25] M.-J. Park, H. S. Cho, T.-S. Yun, S. Byeon, Y. J. Koo, S. Yoon, D. U. Lee, S. Choi, J. Park, J. Lee, K. Cho, J. Moon, B.-K. Yoon, Y.-J. Park, S.-m. Oh, C. K. Lee, T.-K. Kim, S.-H. Lee, H.-W. Kim, Y. Ju, S.-K. Lim, S. G. Baek, K. Y. Lee, S. H. Lee, W. S. We, S. Kim, Y. Choi, S.-H. Lee, S. M. Yang, G. Lee, I.-K. Kim, Y. Jeon, J.-H. Park, J. C. Yun, C. Park, S.-Y. Kim, S. Kim, D.-Y. Lee, S.-H. Oh, T. Hwang, J. Shin, Y. Lee, H. Kim, J. Lee, Y. Hur, S. Lee, J. Jang, J. Chun, and J. Cho, "A 192-Gb 12-High 896-GB/s HBM3 DRAM With a TSV Auto-Calibration Scheme and Machine-Learning-Based Layout Optimization," *IEEE Journal of Solid-State Circuits (JSSC)*, 2022.
- [26] Hugging Face, "Hugging Face Model Hub," <https://huggingface.co>, 2025, accessed on 2025-11-15.
- [27] JEDEC Solid State Technology Association, "JESD238A: High Bandwidth Memory 3 (HBM3) DRAM," JEDEC, Standard, 2023.
- [28] JEDEC Solid State Technology Association, "JESD270-4: High Bandwidth Memory 4 (HBM4) DRAM," JEDEC, Standard, 2025.
- [29] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture," NVIDIA, Tech. Rep., 2020.
- [30] M. B. Sullivan, N. Saxena, M. O'Connor, D. Lee, P. Racunas, S. Hukerikar, T. Tsai, S. K. S. Hari, and S. W. Keckler, "Characterizing and mitigating soft errors in GPU DRAM," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021.
- [31] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture Whitepaper," NVIDIA, Tech. Rep., 2022.
- [32] I. Catalán, J. Flich, and C. Hernández, "Exploiting neural networks bit-level redundancy to mitigate the impact of faults at inference," *The Journal of Supercomputing*, 2025.
- [33] M. Qin, C. Sun, and D. Vucinic, "Robustness of neural networks against storage media errors," *arXiv preprint arXiv:1709.06173*, 2017.
- [34] S.-S. Lee and J.-S. Yang, "Value-aware parity insertion ECC for fault-tolerant deep neural network," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022.
- [35] T. Park, S. Gorgin, D. Kim, J. Shin, M. B. Sullivan, and J. Kim, "PoP-ECC: Robust and flexible error correction against multi-bit upsets in DNN accelerators," in *ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [36] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras, "Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks," in *USENIX Security Symposium (USENIX Security)*, 2019.
- [37] A. Mahmoud, T. Tambe, T. Aloui, D. Brooks, and G.-Y. Wei, "GoldenEye: A platform for evaluating emerging numerical data formats in DNN accelerators," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2022.
- [38] Y. Sun, Z. Coalson, S. Chen, H. Liu, Z. Zhang, S. Hong, B. Fang, and L. Yang, "Demystifying the resilience of large language model inference: An end-to-end perspective," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025.
- [39] M. B. Sullivan, S. K. S. Hari, B. Zimmer, T. Tsai, and S. W. Keckler, "SwapCodes: Error codes for hardware-software cooperative GPU pipeline error detection," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018.
- [40] S. T. Wasim, K. H. Soboka, A. Mahmoud, S. H. Khan, D. Brooks, and G.-Y. Wei, "Hardware resilience properties of text-guided image classifiers," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [41] Z. Chen, G. Li, and K. Pattabiraman, "A low-cost fault corrector for deep neural networks through range restriction," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2021.
- [42] H. Chung, E. Oh, S. Baek, H. Yoon, J. Yoo, S. Lee, Y. Lee, A. Bramhanand, B. Dodds, Y. Zhou, and N. S. Kim, "DRAM fault classification through large-scale field monitoring for robust memory RAS management," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025.
- [43] A. Mahmoud, N. Aggarwal, A. Nobbe, J. R. S. Vicarte, S. V. Adve, C. W. Fletcher, I. Frosio, and S. K. S. Hari, "PyTorchFI: A runtime perturbation tool for DNNs," in *IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2020.
- [44] H. Huang, C. Liu, X. Xue, B. Liu, H. Li, and X. Li, "MRFI: An open-source multiresolution fault injection framework for neural network processing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems (TVLSI)*, 2024.
- [45] T. Xie, J. Zhao, Z. Wan, Z. Zhang, Y. Wang, R. Wang, R. Huang, and M. Li, "ReaLM: Reliable and efficient large language model inference with statistical algorithm-based fault tolerance," in *ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [46] Y. He, P. Balaprakash, and Y. Li, "Fidelity: Efficient resilience analysis framework for deep learning accelerators," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020.
- [47] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "The language model evaluation harness," 07 2024. [Online]. Available: <https://zenodo.org/records/12608602>
- [48] R. Chien, "Cyclic decoding procedures for Bose-Chaudhuri-Hocquenghem codes," *IEEE Transactions on Information Theory (TIT)*, 2003.
- [49] G. Forney, "On decoding BCH codes," *IEEE Transactions on Information Theory (TIT)*, 2003.

- [50] Z. Liang and W. Zhang, "Efficient Berlekamp-Massey algorithm and architecture for Reed-Solomon decoder," *Journal of Signal Processing Systems*, 2017.
- [51] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling," in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2020.
- [52] S. Che, J. W. Sheaffer, M. Boyer, L. G. Szafaryn, L. Wang, and K. Skadron, "A characterization of the Rodinia benchmark suite with comparison to contemporary CMP workloads," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2010.
- [53] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A Revised Benchmark Suite for Scientific and Commercial Throughput Computing," *Center for Reliable and High-Performance Computing*, 2012.
- [54] J. Max, "Quantizing for minimum distortion," *IRE Transactions on Information Theory*, 1960.
- [55] S. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory (TIT)*, 1982.

APPENDIX

A. Abstract

This artifact appendix provides guidelines for reproducing two key results of this paper: (i) the fault-coverage comparison of RangeGuard and prior protection schemes across DRAM fault modes, and (ii) the accuracy degradation of LLM inference under realistic fault injection across a range of BER values, comparing RangeGuard with prior protection schemes. The artifact consists of two workflows: a standalone ECC coverage analyzer and an LLM robustness evaluation pipeline. The following subsections describe how to access the artifact, build the software environment, and run the provided scripts to reproduce the ECC coverage and LLM accuracy results.

B. Artifact check-list (meta-information)

- **Program:** ECC coverage analyzer; LLM fault-injection framework
- **Models:** Llama-3.2-1B; Llama-3.1-8B
- **Data set:** ARC-Easy
- **Run-time environment:** Tested with Python 3.10.12, PyTorch 2.2.0+cu121, Transformers 4.51.0, Datasets 2.16.1, Accelerate 1.2.1, Evaluate 0.4.3, NVIDIA driver 545.23.08 with CUDA 12.3, and a C++17-capable compiler such as g++.
- **Hardware:** CUDA-capable NVIDIA GPU with at least 24 GB of memory for the LLM workflow; CPU for ECC analyzer
- **Metrics:** CE, DUE, SDC probability; accuracy
- **Output:** JSON; CSV; PDF; PNG; log files
- **Experiments:** Table III; Figure 8(b); Figure 8(c)
- **Required disk space:** at least 20 GB for model weights only; additional space is needed for outputs and caches
- **Duration of the experiment:** Approximately 2 days for the full ECC coverage analysis; approximately 5 days for the full LLM robustness evaluation.
- **Publicly available:** Yes
- **Archived:** <https://doi.org/10.5281/zenodo.19815633>

C. Description

The LLM workflow is built on a modified version of `lm-evaluation-harness`, extended with fault-injection support to evaluate the robustness of LLMs to memory errors.

1) *How to access:* The source code, scripts, and guidelines of RangeGuard are publicly available on GitHub (<https://github.com/HanumKo/RangeGuard-AE>) and Zenodo (<https://doi.org/10.5281/zenodo.19815633>).

2) *Hardware dependencies:* The ECC coverage analyzer can be executed on a CPU-only system. The LLM fault-injection framework requires a CUDA-capable NVIDIA GPU with at least 24 GB of memory. The reported experiments use Llama-3.2-1B and Llama-3.1-8B, and the larger model is the main driver of the GPU memory requirement.

3) *Software dependencies:* The required software dependencies and tested versions are summarized in the artifact check-list above.

4) *Data sets:* The LLM evaluation uses the ARC-Easy dataset. No additional dataset is required for the provided workflow.

D. Installation

Before installation, users should prepare a runtime environment that matches the software versions listed in the artifact check-list.

First, clone the public artifact repository and install the Python package used for the LLM fault-injection workflow:

```
$ git clone https://github.com/HanumKo/RangeGuard-AE
$ cd RangeGuard-AE/1_LLM_accuracy/lm-evaluation-harness
$ pip install -e .
```

This step installs the modified `lm-evaluation-harness` used in the artifact.

E. Models

The artifact uses two LLMs: Llama-3.2-1B and Llama-3.1-8B. The model weights are not bundled with the artifact and must be prepared separately before running the LLM workflow. Users may need to request access to the corresponding Meta Llama repositories on Hugging Face before downloading the model weights. Assuming that RangeGuard-AE is the top-level artifact repository, the scripts expect the models to be placed under:

```
RangeGuard-AE/1_LLM_accuracy/pretrained_models/
```

The expected model directories are:

```
RangeGuard-AE/1_LLM_accuracy/pretrained_models/Llama-3.2-1B
RangeGuard-AE/1_LLM_accuracy/pretrained_models/Llama-3.1-8B
```

For example, download the models using the Hugging Face CLI:

```
$ cd RangeGuard-AE/1_LLM_accuracy
$ huggingface-cli login
$ huggingface-cli download meta-llama/Llama-3.2-1B --local-dir pretrained_models/Llama-3.2-1B
$ huggingface-cli download meta-llama/Llama-3.1-8B --local-dir pretrained_models/Llama-3.1-8B
```

Since `lm-evaluation-harness` also supports remote model loading, users may provide a Hugging Face model identifier directly through `--model_args` instead of using a local path.

The corresponding Hugging Face model pages are:

- <https://huggingface.co/meta-llama/Llama-3.2-1B>
- <https://huggingface.co/meta-llama/Llama-3.1-8B>

F. Experiment workflow

The artifact contains two workflows. The first workflow evaluates LLM robustness using the provided fault-injection framework. After preparing the required models, run the LLM evaluation as follows:

```
$ cd RangeGuard-AE/1_LLM_accuracy/lm-evaluation-harness
$ bash run.sh
```

The `run.sh` script can be adapted to multi-GPU environments by modifying the `DEVICE` environment variable. For detailed multi-GPU execution options, users may refer to the original `lm-evaluation-harness` documentation: <https://github.com/EleutherAI/lm-evaluation-harness>.

The second workflow evaluates ECC coverage using the standalone analyzer. First build the analyzer:

```
$ cd RangeGuard-AE/2_ECC_coverage/ecc_coverage
$ bash build_ecc_cov.sh
```

Then run the coverage experiment:

```
$ bash run.sh
```

G. Evaluation and expected results

After the main experiment scripts finish, the generated raw outputs should be post-processed to obtain the final figures and tables.

For the LLM workflow, after running the main evaluation script, aggregate the raw outputs and generate the final plots using:

```
$ cd RangeGuard-AE/1_LLM_accuracy/figure
$ python json2csv.py
$ python make_figure.py
```

This produces aggregated per-model summaries and the final accuracy plots. The expected result is a set of accuracy-versus-BER curves for the evaluated models and protection schemes. These plots correspond to the LLM robustness results reported in Figure 8(b) and Figure 8(c) of the paper.

For the ECC workflow, after running the main coverage experiment, generate the ratio tables from the raw coverage outputs using:

```
$ cd RangeGuard-AE/2_ECC_coverage/ecc_coverage
$ python make_ratio_tables.py
```

This produces summary tables for CE, DUE, and SDC. The expected result is a set of tables whose rows correspond to fault patterns and whose columns correspond to ECC schemes, with each entry reporting the CE, DUE, or SDC ratio. These processed tables correspond to the ECC coverage results reported in Table III of the paper.